Amundsen.io

C. Enrique Israel Torres Mendoza.

Procesos de la estrategia a implementar.

Se pretende trabajar con los diferentes servicios de Amundsen para conectar y visualizar metadatos de un almacenamiento de datos, para ello partimos de un punto en el cual levantamos los contenedores necesarios a través de un comando con podman-compose y un archivo docker.yml

Dentro de nuestra máquina y dentro de la carpeta de nuestro proyecto, en este caso es dentro de la carpeta amundsen, donde se aloja nuestro repositorio, entramos en ese directorio y ejecutamos el siguiente comando

```
podman-compose -f docker-amundsen.yml up
```

Este comando levantará 5 servicios:

- El servicio de Neo4j
- El servicio de Elasticsearch
- El servicio amundsen search, que depende del servicio de Elasticsearch
- El servicio amundsen metadata, que depende de Neo4j
- Y el servicio de amundsen frontend, que depende de los servicios search y metadata de amundsen.

Una vez que se ejecuta el comando ya tendremos nuestros contenedores arriba, con las especificaciones que están dentro del archivo de “docker-amundsen.yml”

Es importante poner :z al final de las siguientes líneas de código y poner al mismo nivel las 4 lineas que se muestran a continuación para que al momento de ejecutar el comando anterior no haya conflictos con los permisos, de no hacer esta instrucción es probable que se nieguen algunos permisos para algunas configuraciones de Neo4j

enriquetorres@localhost:/hom... × enriquetorres@localhost:/hom... × enriquetorres@

```yaml
version: '3'
services:
  neo4j:
      image: neo4j:3.5.26
      container_name: neo4j_amundsen
      environment:
        - NEO4J_AUTH=neo4j/test
      ulimits:
        nofile:
          soft: 40000
          hard: 40000
      ports:
          - 7474:7474
          - 7687:7687
      volumes:
        - ./example/docker/neo4j/conf:/var/lib/neo4j/conf:z
        - ./example/docker/neo4j/plugins:/var/lib/neo4j/plugins:z
        - ./example/backup:/backup:z
        - neo4j_data:/data
      networks:
        - amundsennet
  elasticsearch:
      image: elasticsearch:8.0.0
      container_name: es_amundsen
      ports:
          - 9200:9200
      volumes:
        - es_data:/usr/share/elasticsearch/data
      networks:
        - amundsennet
      ulimits:
        nofile:
          soft: 65536
          hard: 65536
      environment:
        - discovery.type=single-node
        - xpack.security.enabled=false
  amundsensearch:
      image: amundsendev/amundsen-search:4.0.2
      container_name: amundsensearch
      ports:
"docker-amundsen.yml" 87L, 2678B
```

ove the mouse pointer outside or press Ctrl+Alt.

Si se tiene algún error por la denegación de permiso puede tumbar los servicios anteriores y remover los contenedores con el comando “podman-compose -f docker-amundsen.yml down”, realizar la configuración en el servicio de neo4j como se mencionó anteriormente y volver a ejecutar el comando con “up” al final

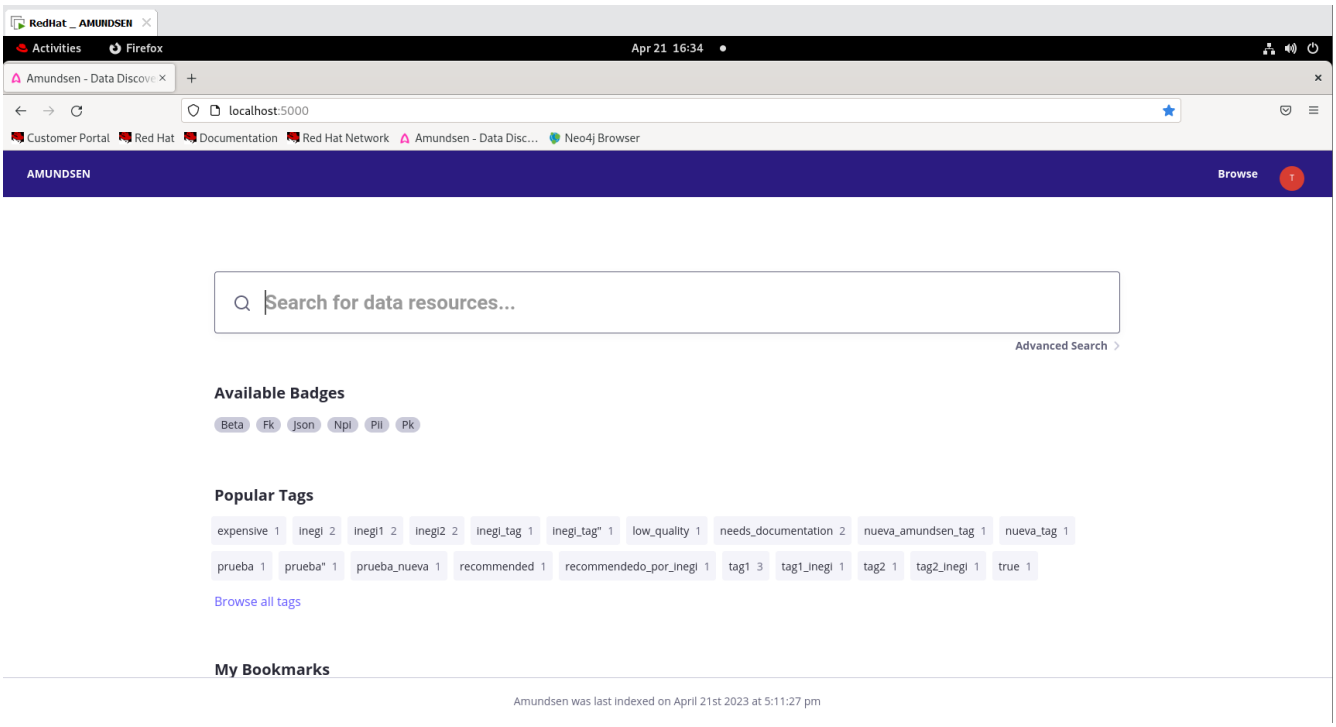
Hecho esto ya están los contenedores funcionando correctamente, pero no se pueden visualizar aun ningún dato ya que aun no se ah hecho la ingesta de datos.

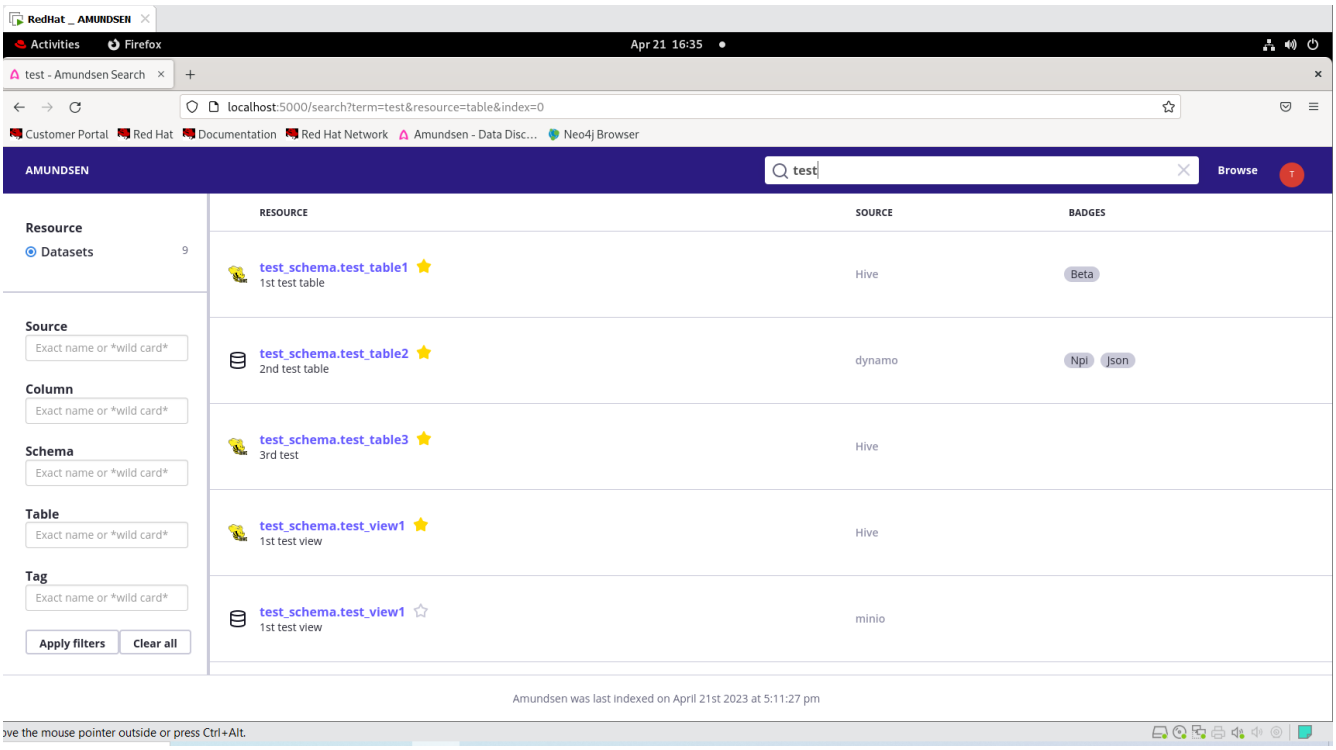
Para ello abrimos otra terminar y nos dirigimos a la carpeta de databuilder y ejecutamos los siguientes comandos.

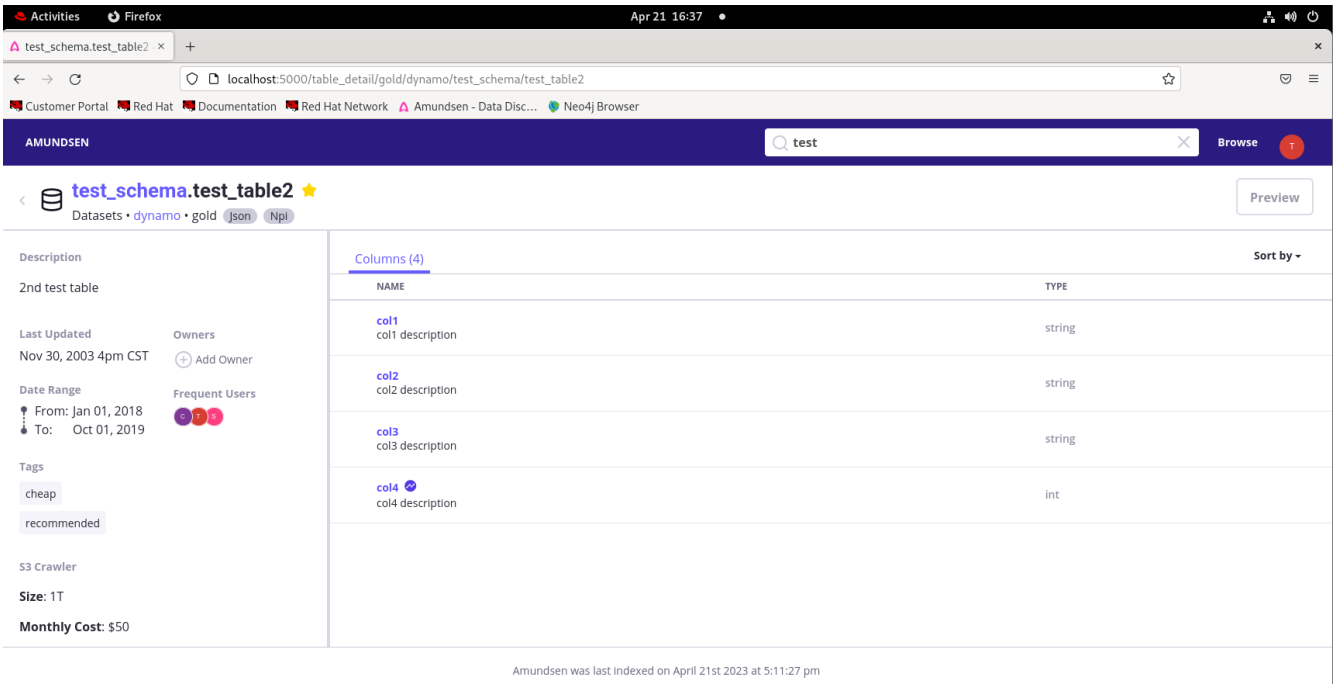
```
$ python3 -m venv venv
$ source venv/bin/activate
$ pip3 install --upgrade pip
$ pip3 install -r requirements.txt
$ python3 setup.py install
$ python3 example/scripts/sample_data_loader.py
```

Todo como modo super usuario.

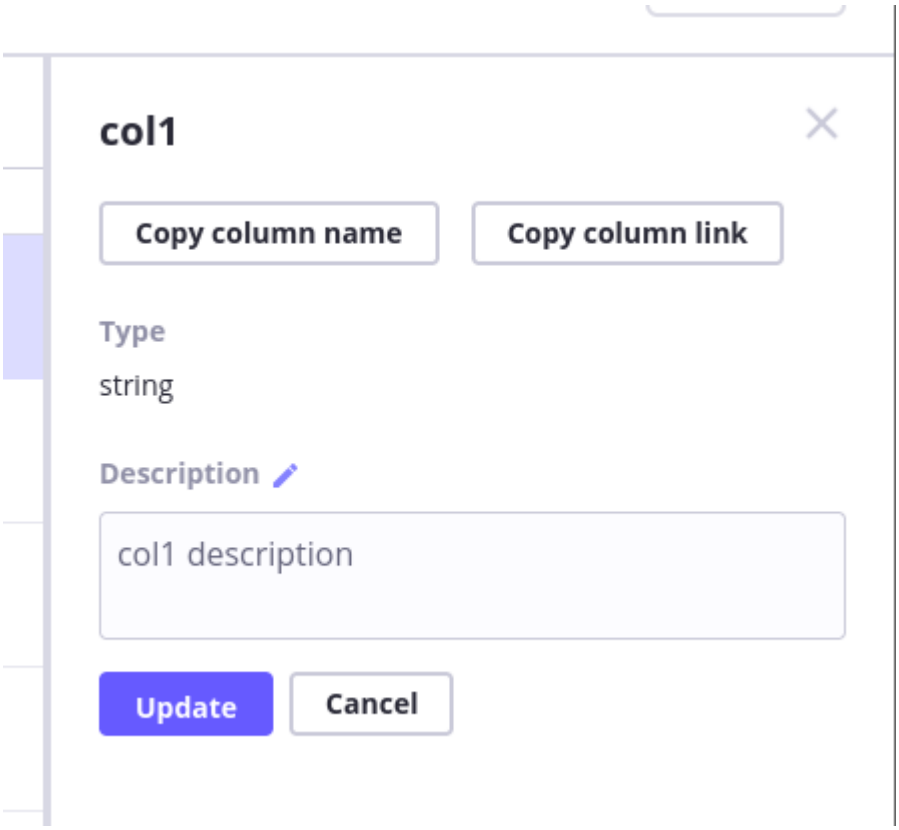
Una vez hecho esto ya podemos visualizar datos en amundsen.





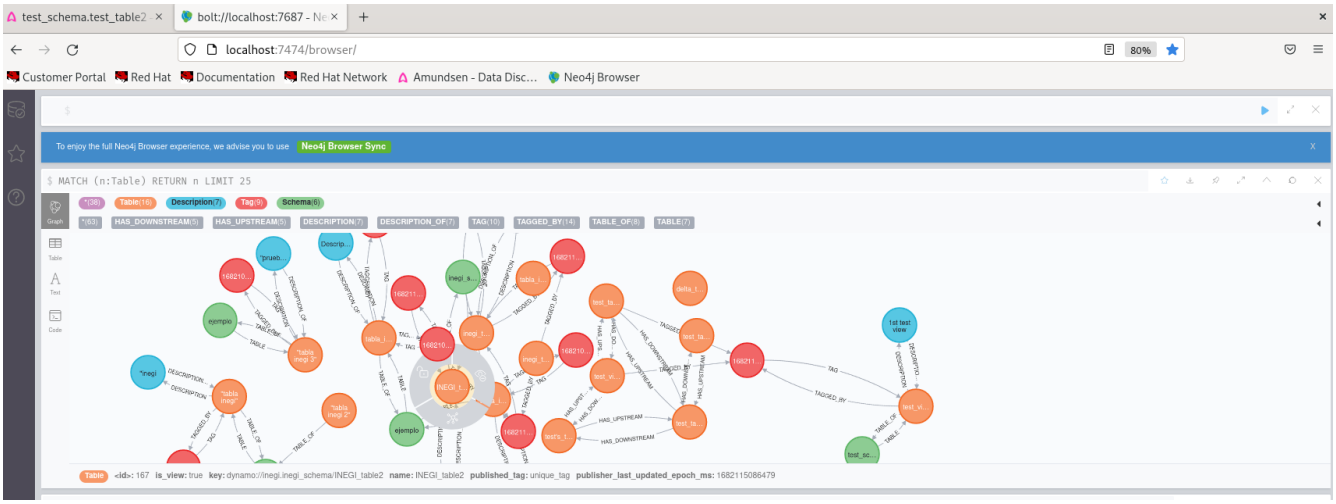


lcid@inegi.org.mx



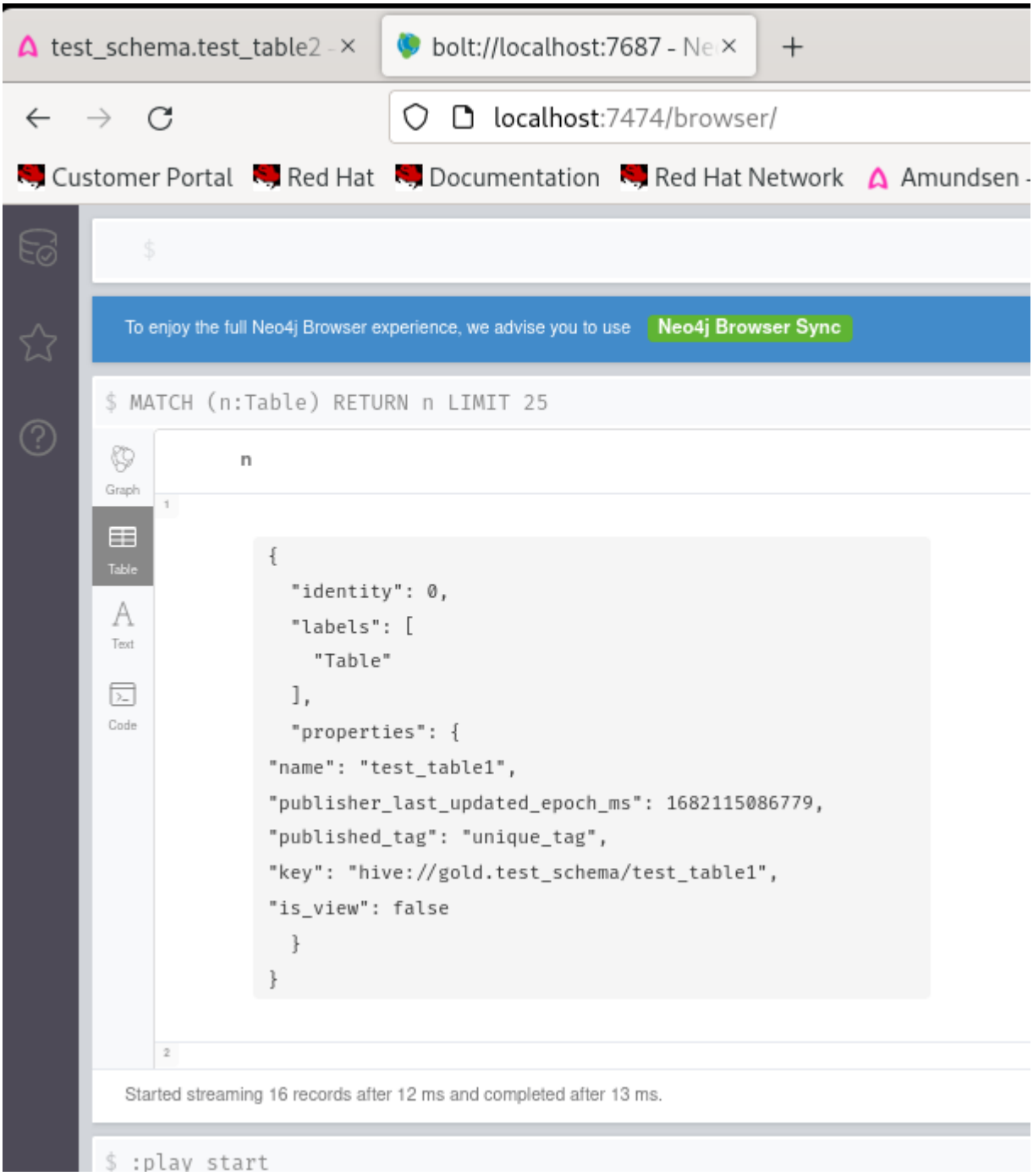
También Podemos la ingesta de datos en Neo4j ingresando a la siguiente URL http://localhost:7474/browser/ y correr el siguiente comando `MATCH (n:Table) RETURN n LIMIT 25`

El resultado debe ser algo como lo siguiente:



lcid@inegi.org.mx

Se obtienen algunas tablas, que se pueden ver representadas en grafos o también en tablas, en formato json o yaml.



Amundsen utiliza un cuarto servicio que es el llamado “databuilder”

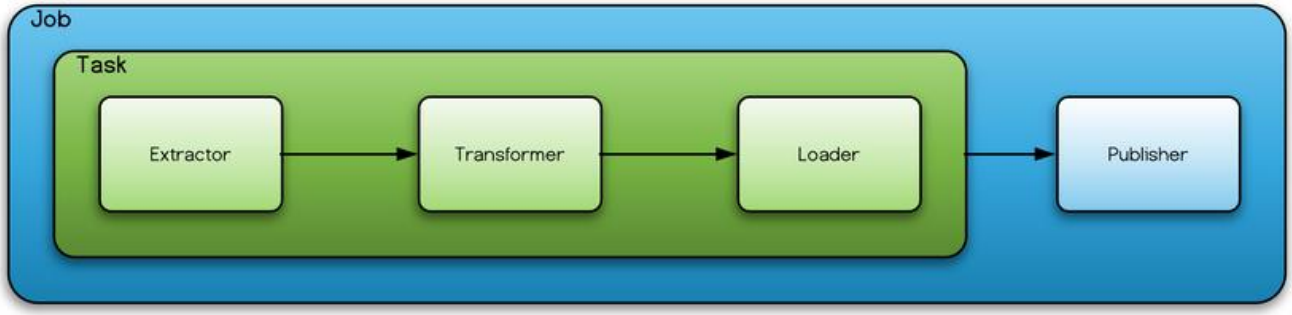
Amundsen Databuilder es una biblioteca de ingesta de datos inspirada en Apache Gobblin. Podría usarse en un marco de orquestación (por ejemplo, Apache Airflow) para generar datos de Amundsen. Puede usar la biblioteca con un script de python adhoc o dentro de un Apache Airflow DAG.

CONCEPT.

Extracción, transformación y carga (ETL) es el proceso consistente en combinar datos de diferentes orígenes un gran repositorio central llamado almacenamiento de datos.

El trabajo de ETL consiste en la extracción de registros de la fuente, transformar registros, si es necesario, y cargar registros en el receptor. Amundsen Databuilder es un marco ETL para Amundsen y hay componentes correspondientes para ETL llamados Extractor, Transformer y Loader que se ocupan de la operación de nivel de registro. Un componente llamado “Task” controla estos tres componentes. El “Job” es el componente de más alto nivel en Databuilder que controla la tarea y el publicador y es el que usa el cliente para iniciar el trabajo de ETL.

En Databuilder, cada componente está altamente modularizado y cada componente utiliza una configuración basada en el espacio de nombres, configuración HOCON, lo que lo hace altamente reutilizable y conectable.



En Databuilder, cada componente está altamente modularizado y cada componente utiliza una configuración basada en el espacio de nombres, configuración HOCON, lo que lo hace altamente reutilizable y conectable.

Extractor: Un extractor extrae registros de la fuente. Esto no significa necesariamente que solo admita el patrón de extracción en ETL. Por ejemplo, la extracción de registros del bus de mensajería lo convierte en un patrón de inserción en ETL.

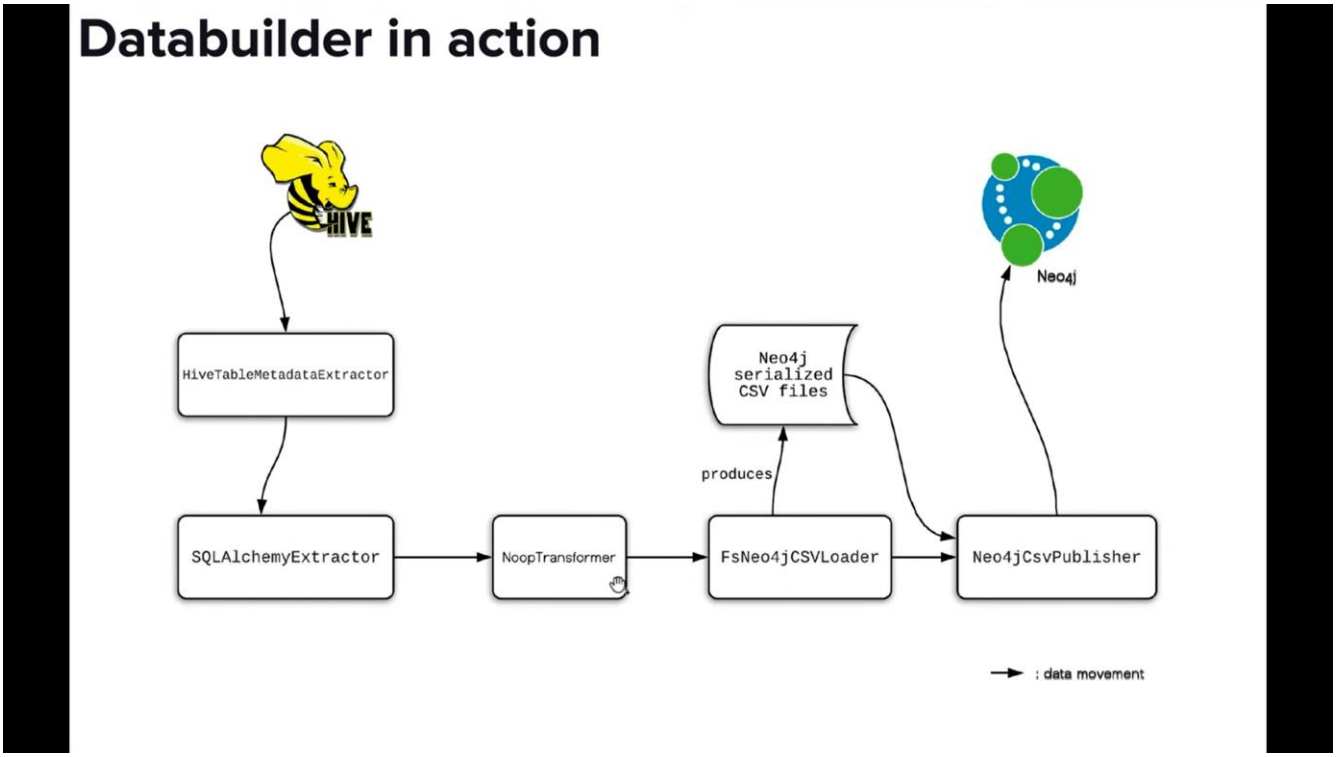
Transformer: Un transformador toma un registro de un extractor o de otros transformadores (a través de Transformador encadenado / ChainedTransformer) para transformar el registro.

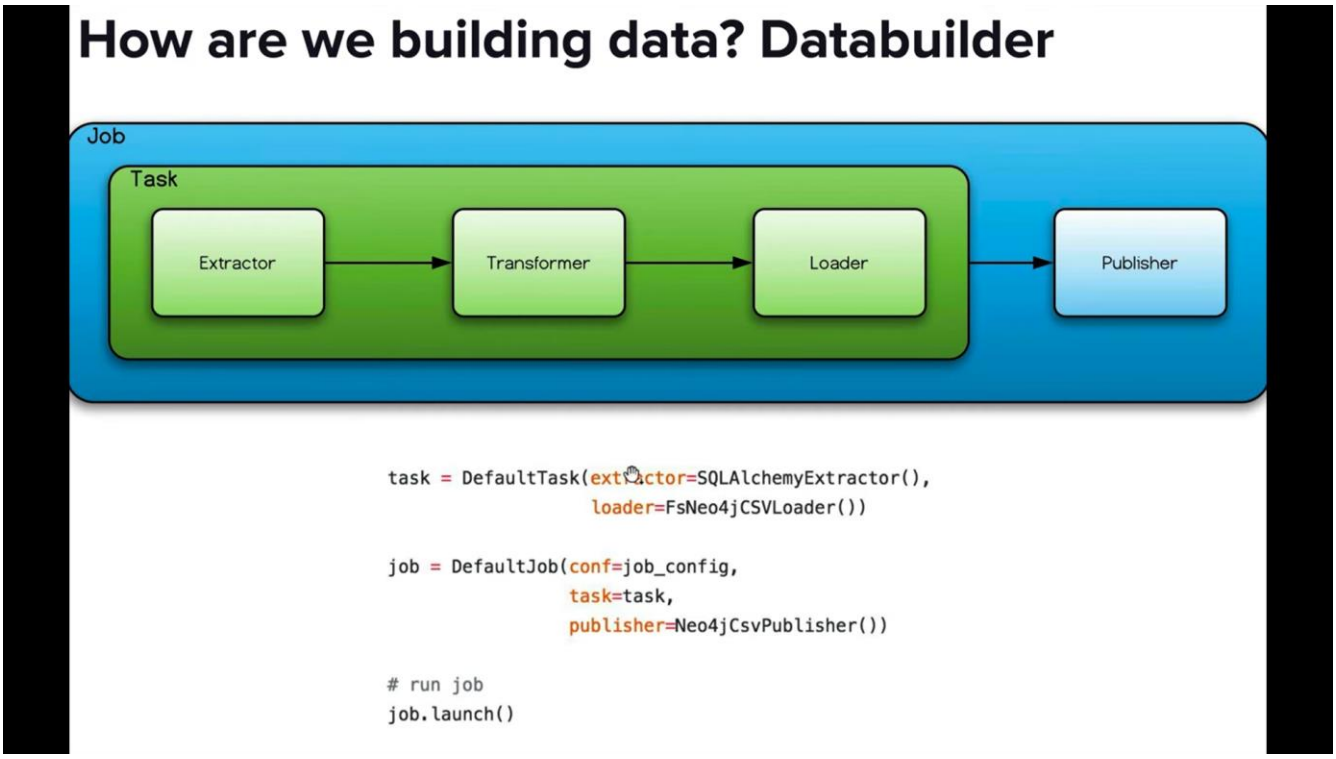
Loader: Un cargador toma un registro de un transformador o de un extractor directamente y lo carga en un sumidero o en un área de preparación. Como la carga opera a un nivel de registro (record-level), no es capaz de soportar la atomicidad.

Task. Una tarea organiza un extractor, un transformador y un cargador para realizar una operación de nivel de registro.

Publisher: El uso común del publicador es soportar la atomicidad en el nivel de trabajo y/o soportar fácilmente la carga a granel en el fregadero.

Job: Un "Job" es el componente de más alto nivel en Databuilder y organiza una tarea y, si lo hay, un publicador.





Amundsen databuilder es el servicio que indexa los metadatos desde algún origen, se utilizan diferentes funciones para hacer que los metadatos de algún recurso puedan ser visualizados estas funciones hacen llamadas a los diferentes extractores, transformadores y cargadores, se declaran las tareas que controlan estos tres componentes, el publicador es controlado por el job, el componente de más alto nivel.

Hay que notar que cuando corremos el comando `python3 example/scripts/sample_data_loader.py` dentro de databuilder el archivo "sample_data_loader.py" es el que trabaja con archivos csv junto de funciones job, para realizar el trabajo ETL, con lo cual nosotros podemos hacer algunas pruebas insertando nuestros propios datos. Las configuraciones se muestran a continuación, se creó un archivo en la carpeta de scripts llamado mis_datos_inegi.py

Que contiene casi todo igual que el archivo "sample_data_loader.py"

Este tiene algunas funciones que trabajan con archivos csv, en nuestro archivo cambiamos el nombre de los dos archivos que se muestran ya que también los vamos a crear personalmente y le pondremos información nuestra

```
if __name__ == "__main__":
    # Uncomment next line to get INFO level logging
    # logging.basicConfig(level=logging.INFO)

    run_table_column_job('example/sample_data/mi_tabla_inegi.csv', 'example/sample_data/inegi_col.csv')
    run_table_badge_job('example/sample_data/mi_tabla_inegi.csv', 'example/sample_data/sample_badges.csv')
    run_table_lineage_job('example/sample_data/sample_table_lineage.csv')
    run_column_lineage_job('example/sample_data/sample_column_lineage.csv')
```

Estos archivos los debemos crear con el comando en Linux “touch nombre_del_archivo.extension” dentro de la carpeta sample_data

```
[root@localhost sample_data]# ls -l
total 104
drwxr-xr-x. 2 root root   47 Apr 12 18:21 dbt
-rw-r--r--. 1 root root  336 Apr 21 14:06 inegi_col.csv
-rw-r--r--. 1 root root  316 Apr 21 14:01 mi_tabla_inegi.csv
drwxr-xr-x. 2 root root   46 Apr 12 18:21 openlineage
drwxr-xr-x. 2 root root   31 Apr 12 18:21 pandas_profiling
-rw-r--r--. 1 root root  210 Apr 12 18:21 sample_application.csv
-rw-r--r--. 1 root root  160 Apr 12 18:21 sample_badges.csv
-rw-r--r--. 1 root root 1046 Apr 12 18:21 sample_col.csv
-rw-r--r--. 1 root root  284 Apr 12 18:21 sample_column_lineage.csv
-rw-r--r--. 1 root root  998 Apr 12 18:21 sample_column_usage.csv
-rw-r--r--. 1 root root 1065 Apr 12 18:21 sample_dashboard_base.csv
-rw-r--r--. 1 root root  447 Apr 12 18:21 sample_dashboard_last_execution.csv
-rw-r--r--. 1 root root  185 Apr 12 18:21 sample_dashboard_last_modified.csv
-rw-r--r--. 1 root root  189 Apr 12 18:21 sample_dashboard_owner.csv
-rw-r--r--. 1 root root  416 Apr 12 18:21 sample_dashboard_query.csv
-rw-r--r--. 1 root root  261 Apr 12 18:21 sample_dashboard_table.csv
-rw-r--r--. 1 root root  211 Apr 12 18:21 sample_dashboard_usage.csv
-rw-r--r--. 1 root root   91 Apr 12 18:21 sample_schema_description.csv
-rw-r--r--. 1 root root  137 Apr 12 18:21 sample_source.csv
-rw-r--r--. 1 root root  898 Apr 12 18:21 sample_table_column_stats.csv
-rw-r--r--. 1 root root  521 Apr 21 14:19 sample_table.csv
-rw-r--r--. 1 root root  145 Apr 12 18:21 sample_table_last_updated.csv
-rw-r--r--. 1 root root  475 Apr 12 18:21 sample_table_lineage.csv
-rw-r--r--. 1 root root  162 Apr 12 18:21 sample_table_owner.csv
-rw-r--r--. 1 root root  545 Apr 12 18:21 sample_table_programmatic_source.csv
-rw-r--r--. 1 root root  191 Apr 12 18:21 sample_table_report.csv
-rw-r--r--. 1 root root   47 Apr 12 18:21 sample_tags.csv
-rw-r--r--. 1 root root  570 Apr 12 18:21 sample_user.csv
-rw-r--r--. 1 root root  424 Apr 12 18:21 sample_watermark.csv
[root@localhost sample_data]#
```

Los cuales se configuran con información como la siguiente.

```
database,cluster,schema,name,description,tags,is_view,description_source
minio,inegi,inegi_schema,inegi_table1,"1st inegi table","tag1_inegi,tag2_INEGI",false,
dynamo,inegi,inegi_schema,INEGI_table2,"2nd inegi test table",recommendedo_por_inegi,false,
minio,infra,test_schema,test_view1,"1st test view","tag1",true,
~
```

"mi_tabla_inegi.csv"

```
name,description,col_type,sort_order,database,cluster,schema,table_name,badges
id,"id inegi description","int",1,minio,inegi,inegi_schema,test_table1,PK
nombre,"nombre de la description","string",2,minio,inegi,inegi_schema,test_table1,PII
direccion,"direccion de la description","string",3,minio,inegi,inegi_schema,test_table1,"FK,PII"
~
```

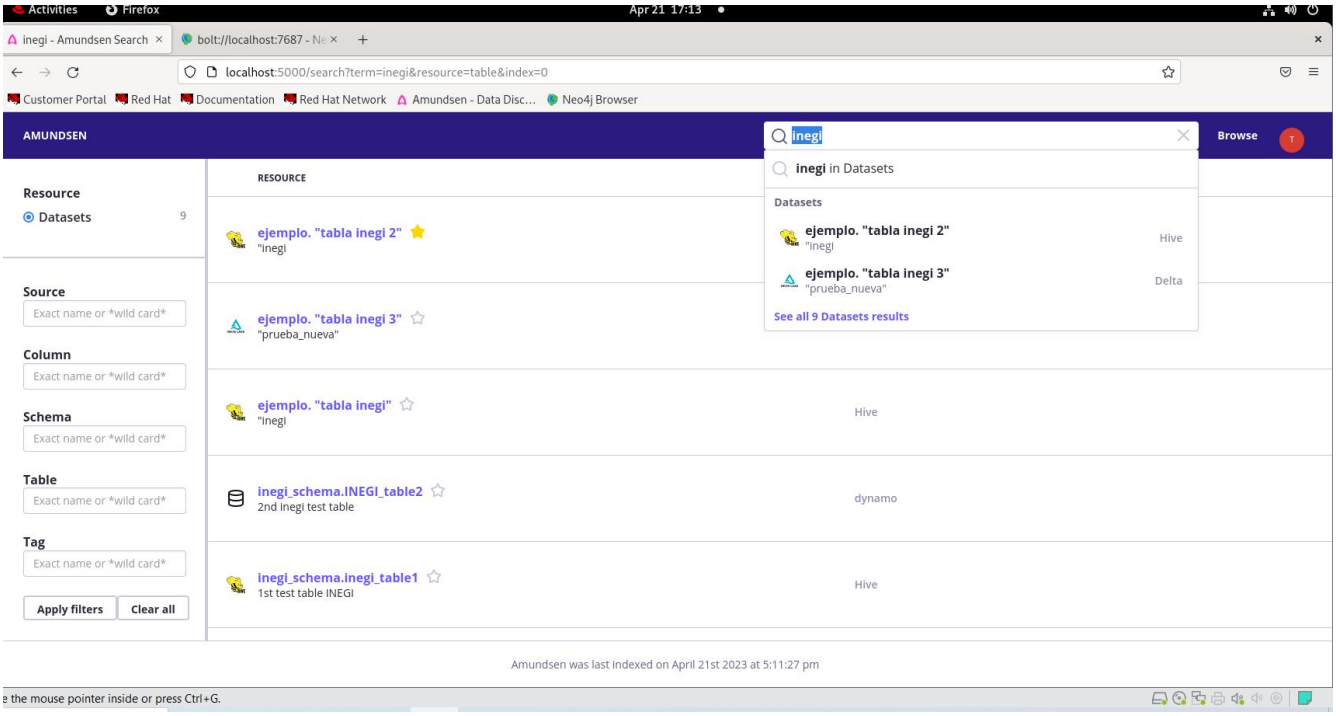
"inegi_col.csv"

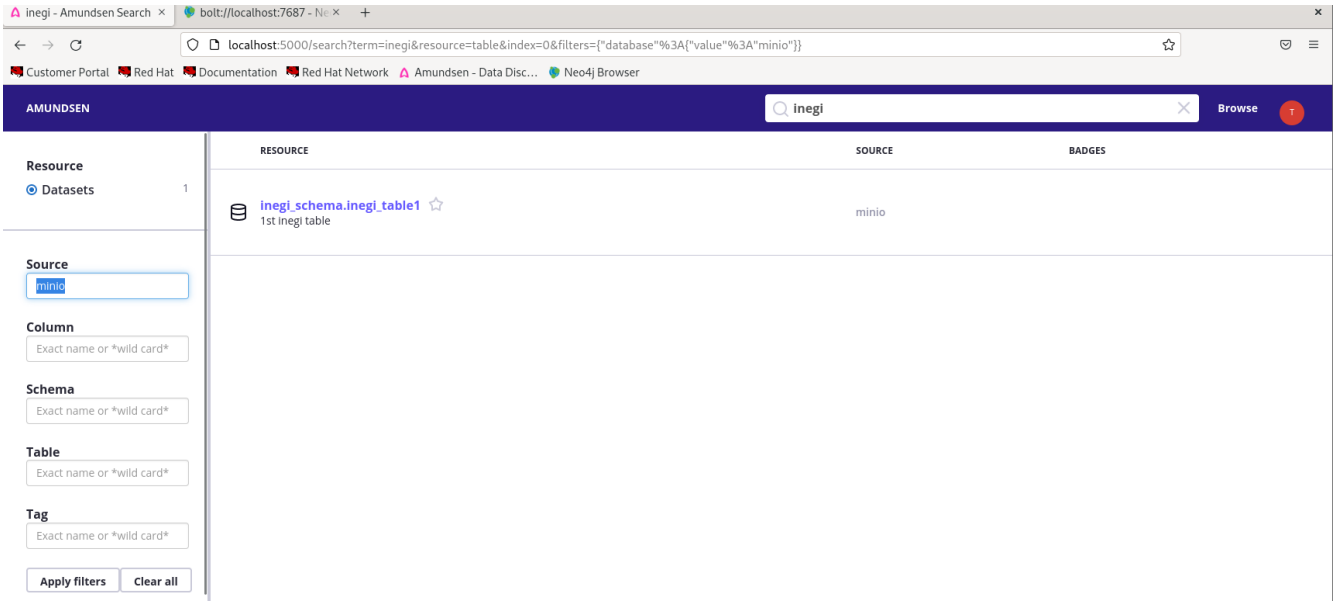
Teniendo esta configuración regresamos a la carpeta de databuilder y vamos a ejecutar el comando que se usó anteriormente pero ahora llamando al archivo que creamos, dentro de la carpeta scripts.

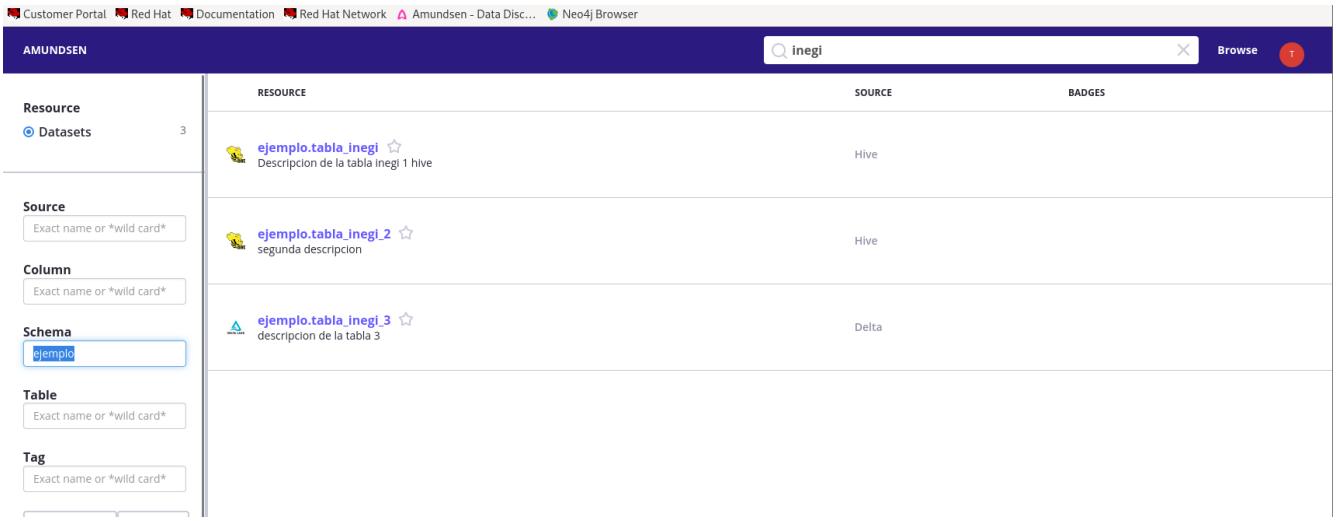
```
[root@localhost scripts]# ls -l
total 272
-rw-r--r--. 1 root root 20257 Apr 21 16:09 mis_datos_inegi.py
-rw-r--r--. 1 root root  3815 Apr 12 18:21 sample_atlas_search_extractor.py
-rw-r--r--. 1 root root  2835 Apr 12 18:21 sample_bigquery_metadata.py
-rw-r--r--. 1 root root  2862 Apr 12 18:21 sample_bq_usage_loader.py
-rw-r--r--. 1 root root 22109 Apr 12 18:21 sample_data_loader_atlas.py
-rw-r--r--. 1 root root 16633 Apr 12 18:21 sample_data_loader_mysql.py
-rw-r--r--. 1 root root 18743 Apr 12 18:21 sample_data_loader_neptune.py
-rw-r--r--. 1 root root 21149 Apr 12 18:21 sample_data_loader.py
-rw-r--r--. 1 root root  7755 Apr 12 18:21 sample_db2_data_loader.py
-rw-r--r--. 1 root root  8101 Apr 12 18:21 sample_dbt_loader.py
-rw-r--r--. 1 root root  3305 Apr 12 18:21 sample_deltalake_metadata.py
-rw-r--r--. 1 root root  7599 Apr 12 18:21 sample_dremio_data_loader.py
-rw-r--r--. 1 root root  6395 Apr 12 18:21 sample_feast_loader.py
-rw-r--r--. 1 root root  5233 Apr 12 18:21 sample_glue_loader.py
-rw-r--r--. 1 root root  8569 Apr 12 18:21 sample_mssql_metadata.py
-rw-r--r--. 1 root root  7725 Apr 12 18:21 sample_mysql_loader.py
-rw-r--r--. 1 root root  2043 Apr 12 18:21 sample_neptune_data_cleanup_job.py
-rw-r--r--. 1 root root  7655 Apr 12 18:21 sample_oracle_loader.py
-rw-r--r--. 1 root root  2251 Apr 12 18:21 sample_pandas_profiling_column_stats_loader.py
-rw-r--r--. 1 root root  7912 Apr 12 18:21 sample_postgres_loader.py
-rw-r--r--. 1 root root  2164 Apr 12 18:21 sample_search_metadata_task.py
-rw-r--r--. 1 root root  8501 Apr 12 18:21 sample_snowflake_data_loader.py
-rw-r--r--. 1 root root  4621 Apr 12 18:21 sample_snowflake_table_usage.scala
-rw-r--r--. 1 root root 17842 Apr 12 18:21 sample_tableau_data_loader.py
-rw-r--r--. 1 root root  7943 Apr 12 18:21 sample_teradata_loader.py
-rw-r--r--. 1 root root  8717 Apr 12 18:21 sample_vertica_loader.py
[root@localhost scripts]#
```

```
(venv) [root@localhost databuilder]# python3 example/scripts/mis_datos_inegi.py
/home/enriquetorres/amundsen/databuilder/example/scripts/mis_datos_inegi.py:42: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_base().
(deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
  Base = declarative_base()
WARNING:databuilder.publisher.elasticsearch_publisher:ElasticsearchPublisher is being deprecated in favor of using SearchMetadatatoElasticasearchTask           which publishes ES metadata
with mappings compatible with amundsensearch >= 4.0.0
WARNING:databuilder.publisher.elasticsearch_publisher:ElasticsearchPublisher is being deprecated in favor of using SearchMetadatatoElasticasearchTask           which publishes ES metadata
with mappings compatible with amundsensearch >= 4.0.0
WARNING:databuilder.publisher.elasticsearch_publisher:ElasticsearchPublisher is being deprecated in favor of using SearchMetadatatoElasticasearchTask           which publishes ES metadata
with mappings compatible with amundsensearch >= 4.0.0
(venv) [root@localhost databuilder]#
```

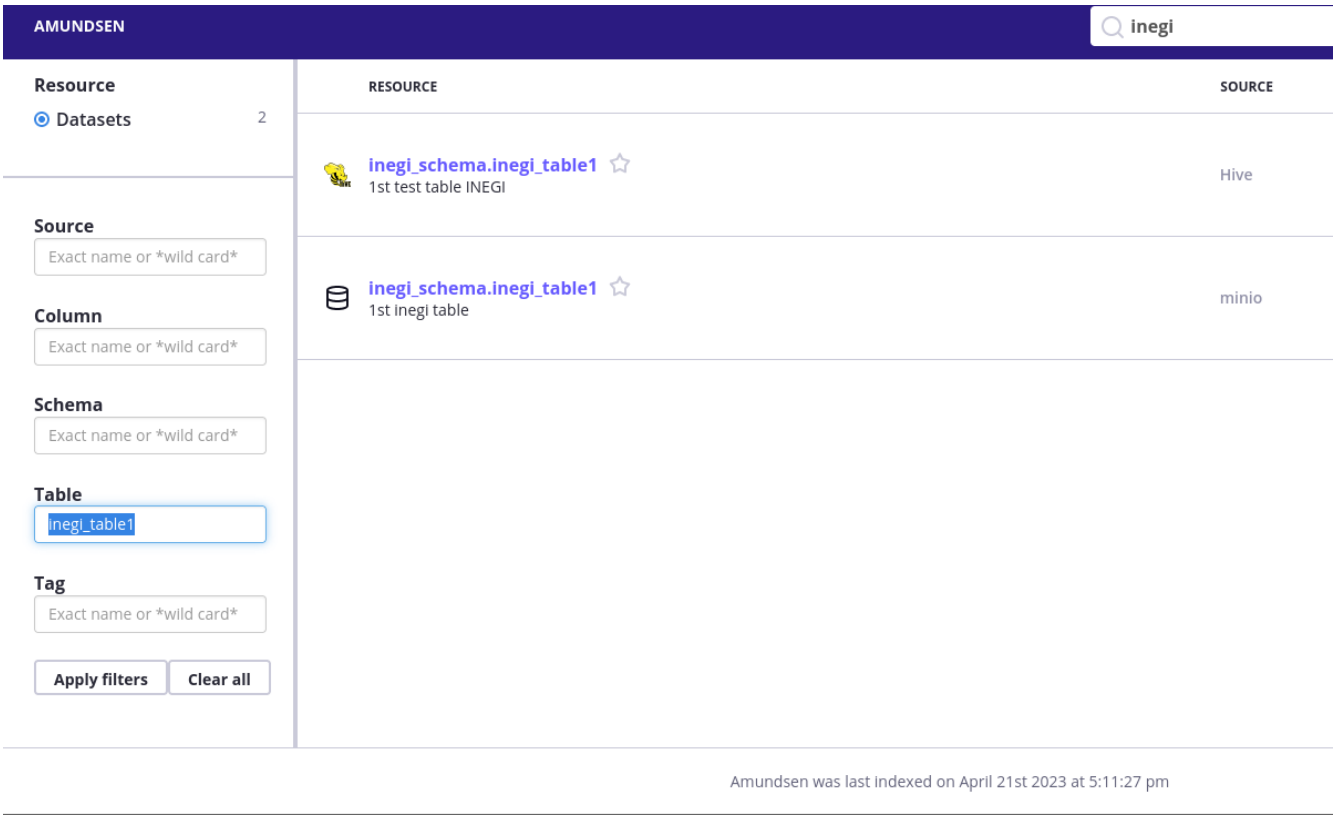
De vuelta en amundsen podemos ahora buscar diferentes términos que nosotros mismos declaramos en los archivos csv que creamos con anterioridad y la ingesta de estos nuevos datos ya se habrá realizado tanto en Neo4j como en amundsen, y los podemos buscar gracias también a los servicios de búsqueda y Elasticsearch.

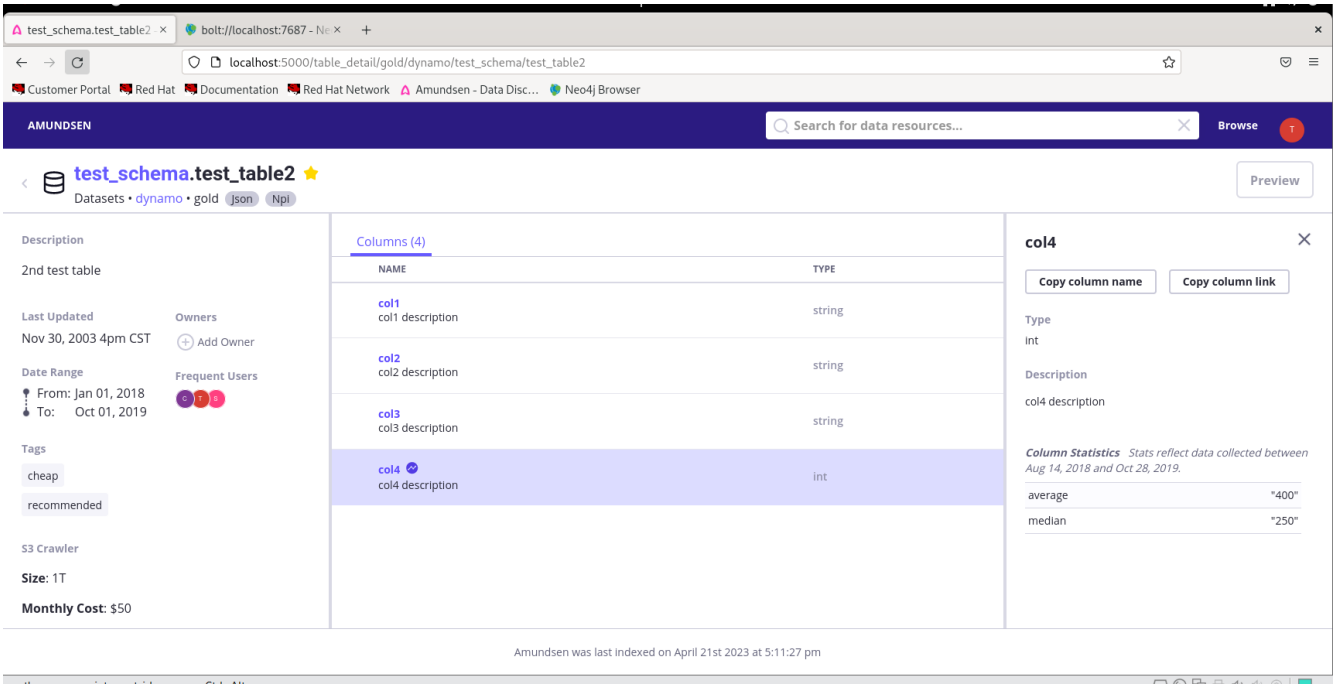






lcid@inegi.org.mx





lcid@inegi.org.mx

De este modo se entiende mejor a nivel de ejemplo cómo es que trabaja el databuilder para indexar metadatos en amundsen, se pretende avanzar en la investigación para realizar una configuración personalizada en Python para poder conectar con un almacén de datos en S3 y extraer metadatos e información de tablas de bases de datos que se encuentren en ese almacén de datos, y poder realizar análisis de datos desde la interfaz de amundsen con la ayuda de sus diferentes servicios propios e independientes.

Gracias.

Bibliografía.

[1] https://github.com/amundsen-io/amundsen/blob/main/docker-amundsen.yml
[2] https://www.amundsen.io/amundsen/databuilder/#list-of-extractors
[3] https://www.amundsen.io/amundsen/installation/
[4] https://github.com/amundsen-io/amundsen/blob/a99fc0d4dbc5b90d3d4d61e1270a445e3c0072d4/databuilder/example/scripts/sample_data_loader.py#L137
[5] https://github.com/amundsen-io/amundsen/blob/a99fc0d4dbc5b90d3d4d61e1270a445e3c0072d4/databuilder/example/sample_data/sample_table.csv
[6] https://github.com/amundsen-io/amundsen/blob/a99fc0d4dbc5b90d3d4d61e1270a445e3c0072d4/databuilder/example/sample_data/sample_col.csv
[7] https://www.youtube.com/watch?v=s-kw9qDQ6jA
[8] https://www.youtube.com/watch?v=5Mmvvyxcnww&list=PL0UJdxehTNlKnGU_h7k2fzJyvAiufeh1U&index=3
[9] https://www.youtube.com/watch?v=Im2V-47S2YU
[10] https://www.youtube.com/watch?v=N_VJZkdIdGU&list=PLkjz14EiPnOB_Zgr_hHlriHNY3JpBcW0B&index=8
[11] https://www.youtube.com/watch?v=ehwp4xUUgEA&t=1s